

CSA0918 – Assignment 3
Name: J O Joanna Divine

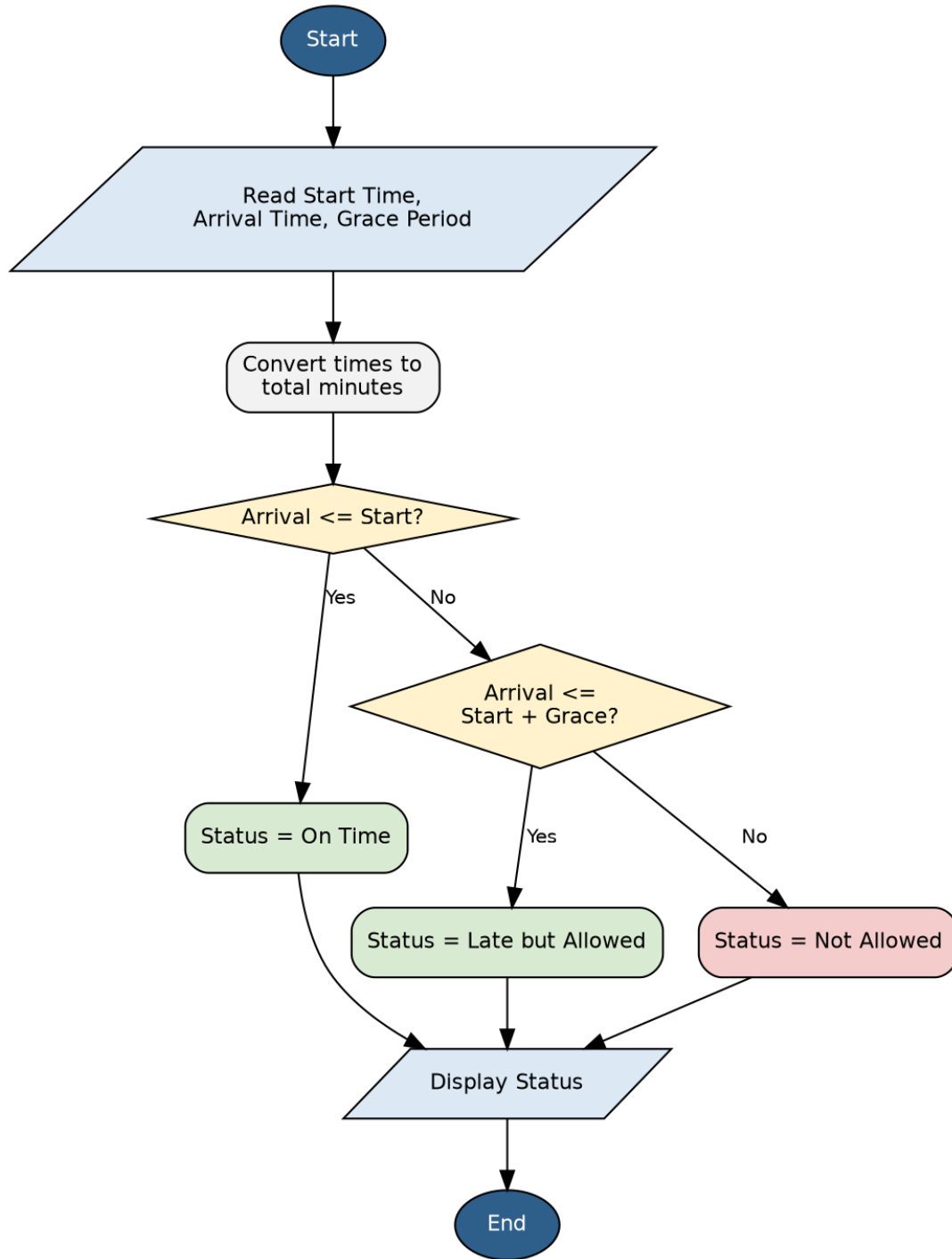
Question 1:

Read an exam start time, arrival time, and grace period in minutes. Determine whether the student is On Time, Late but Allowed, or Not Allowed.

Algorithm

1. Start
2. Read exam start time, student arrival time, and grace period (in minutes).
3. Convert start time and arrival time into total minutes from midnight.
4. If arrival time $\leq$ start time $\rightarrow$ Status = "On Time".
5. Else if arrival time $\leq$ (start time + grace period) $\rightarrow$ Status = "Late but Allowed".
6. Else $\rightarrow$ Status = "Not Allowed".
7. Display the status.
8. Stop

Flowchart



Java Code

```
import java.util.Scanner;

public class Q1_ExamStatus {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.print("Enter exam start time (HH:MM, 24-hr): ");
String startTime = sc.nextLine();

System.out.print("Enter student arrival time (HH:MM, 24-
hr): ");
String arrivalTime = sc.nextLine();

System.out.print("Enter grace period (in minutes): ");
int gracePeriod = sc.nextInt();

// Convert HH:MM to total minutes for comparison
int startMinutes = toMinutes(startTime);
int arrivalMinutes = toMinutes(arrivalTime);
int allowedMinutes = startMinutes + gracePeriod;

String status;
if (arrivalMinutes <= startMinutes) {
    status = "On Time";
} else if (arrivalMinutes <= allowedMinutes) {
    status = "Late but Allowed";
} else {
    status = "Not Allowed";
}

System.out.println("\nExam Start Time : " + startTime);
System.out.println("Arrival Time      : " + arrivalTime);
System.out.println("Grace Period      : " + gracePeriod + "
minutes");
System.out.println("Status          : " + status);
}

// Helper method to convert HH:MM into total minutes
private static int toMinutes(String time) {
    String[] parts = time.split(":");
    int hours = Integer.parseInt(parts[0]);
    int minutes = Integer.parseInt(parts[1]);
    return (hours * 60) + minutes;
}
}

```

Sample Output

```
Enter exam start time (HH:MM, 24-hr): 09:00
Enter student arrival time (HH:MM, 24-hr): 09:07
Enter grace period (in minutes): 10
```

```
Exam Start Time : 09:00
Arrival Time      : 09:07
Grace Period      : 10 minutes
Status           : Late but Allowed
```

Other cases: arrival 08:55 → On Time | arrival 09:25 (with 10-min grace) → Not Allowed

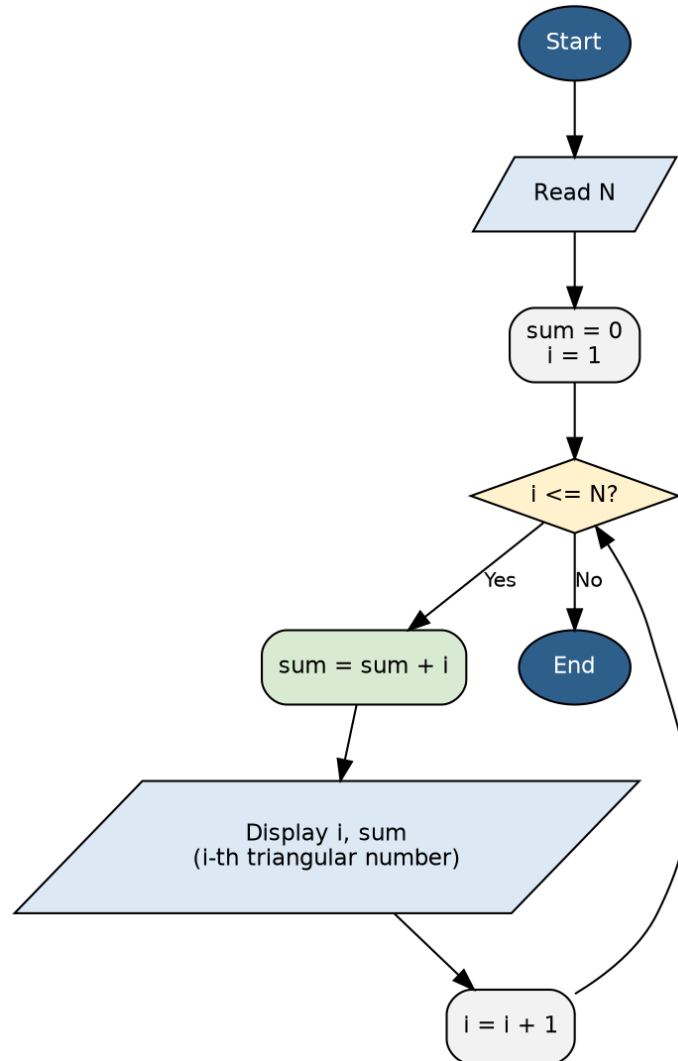
Question 2:

Read an integer N and generate the first N triangular numbers using cumulative addition.

Algorithm

1. Start
2. Read N.
3. Initialise $\text{sum} = 0$ and $i = 1$.
4. Repeat while $i \leq N$:
 5. a. $\text{sum} = \text{sum} + i$ (cumulative addition)
 6. b. Display i and sum as the i -th triangular number
 7. c. $i = i + 1$
8. Stop

Flowchart



Java Code

```
import java.util.Scanner;

public class Q2_TriangularNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter N: ");
        int n = sc.nextInt();

        int sum = 0;
        System.out.println("\nFirst " + n + " Triangular
Numbers:");

        for (int i = 1; i <= n; i++) {
            sum = sum + i;           // cumulative addition
            System.out.println("T(" + i + ") = " + sum);
        }
    }
}
```

Sample Output

Enter N: 6

First 6 Triangular Numbers:

T(1) = 1

T(2) = 3

T(3) = 6

T(4) = 10

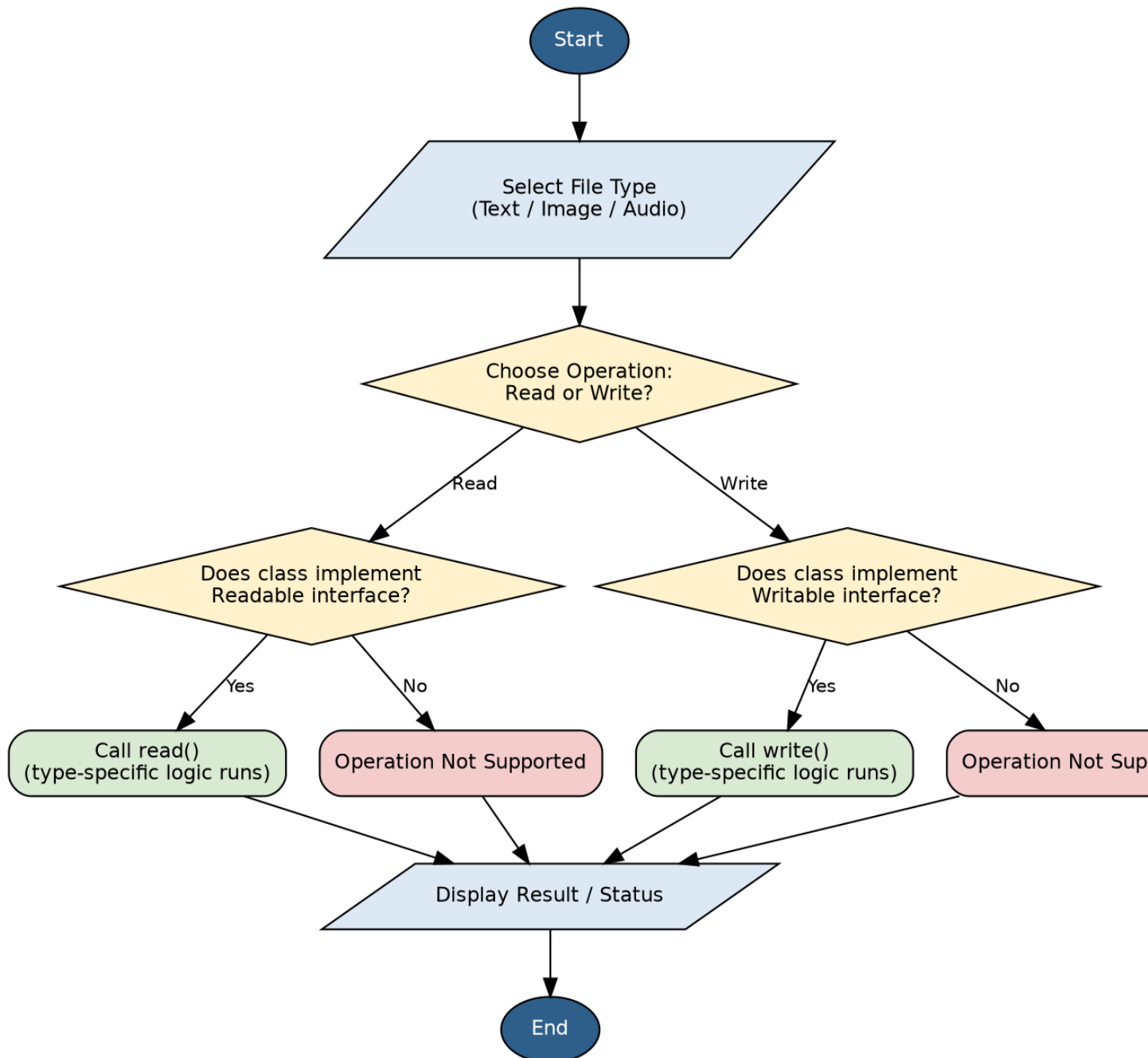
T(5) = 15

T(6) = 21

Question 3:

Design a file-processing flowchart with Readable and Writable behaviours. Explain how Java interfaces could define read() and write() operations for different file types.

Flowchart



Explanation / OOP Mapping

Two interfaces, Readable and Writable, are defined to declare a read() and a write() method respectively. Any file type that supports a given behaviour implements the matching interface, so the capability of a class is decided by which interfaces it implements rather than by its class hierarchy.

TextFile and ImageFile implement both interfaces since they can be read and written, while LogFile implements only Writable because it is an append-only file that is never read back. Each class provides its own version of read()/write(), so the same method call behaves differently depending on the actual file type — this is runtime polymorphism through interfaces.

This design keeps read and write capabilities independent and pluggable: a new file type (e.g. AudioFile) can be added later by simply implementing whichever interfaces apply to it, without changing any existing class or the interfaces themselves.

Java Code

```
// Interface defining read behaviour for any readable file type
interface Readable {
    void read();
}

// Interface defining write behaviour for any writable file type
interface Writable {
    void write();
}

// TextFile supports both reading and writing
class TextFile implements Readable, Writable {
    private String name;
    TextFile(String name) { this.name = name; }

    public void read() {
        System.out.println("Reading text file: " + name + " (line
by line)");
    }
    public void write() {
        System.out.println("Writing text file: " + name + "
(character stream)");
    }
}

// ImageFile supports reading and writing using binary/pixel data
class ImageFile implements Readable, Writable {
    private String name;
    ImageFile(String name) { this.name = name; }

    public void read() {
```

```

        System.out.println("Reading image file: " + name + "
(decoding pixel data)");
    }
    public void write() {
        System.out.println("Writing image file: " + name + "
(encoding pixel data)");
    }
}

// LogFile is write-only (append-only log, never read back by this
system)
class LogFile implements Writable {
    private String name;
    LogFile(String name) { this.name = name; }

    public void write() {
        System.out.println("Writing log file: " + name + "
(appending entry)");
    }
}

public class Q3_FileProcessing {
    public static void main(String[] args) {
        Readable[] readableFiles = { new TextFile("notes.txt"), new
ImageFile("photo.png") };
        Writable[] writableFiles = { new TextFile("notes.txt"), new
ImageFile("photo.png"), new LogFile("system.log") };

        System.out.println("--- Read Operations ---");
        for (Readable f : readableFiles) {
            f.read();
        }

        System.out.println("\n--- Write Operations ---");
        for (Writable f : writableFiles) {
            f.write();
        }
    }
}

```

Sample Output

```
--- Read Operations ---
```

```
Reading text file: notes.txt (line by line)
Reading image file: photo.png (decoding pixel data)
```

```
--- Write Operations ---
```

```
Writing text file: notes.txt (character stream)
Writing image file: photo.png (encoding pixel data)
Writing log file: system.log (appending entry)
```